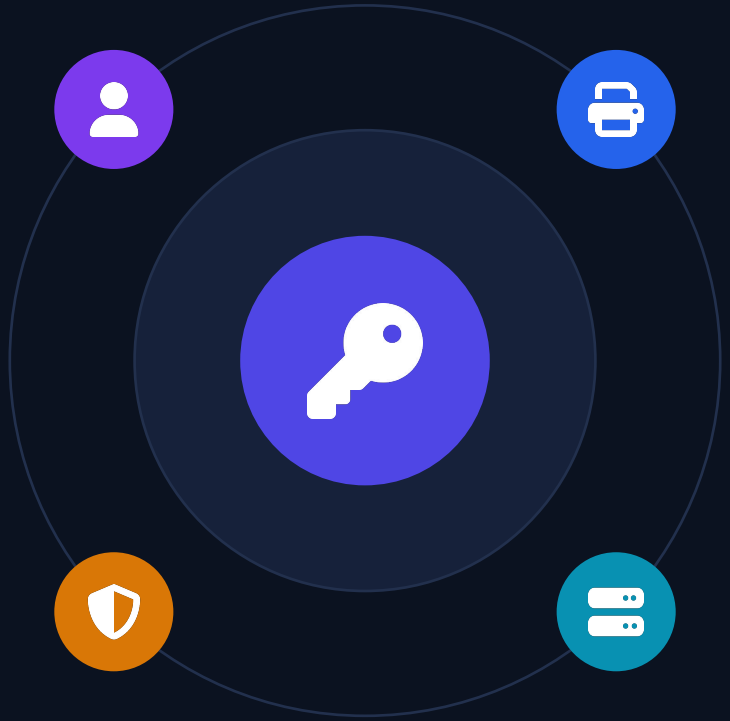
ИНТЕРНЕТ-ТЕХНОЛОГИИ · СПбПУ

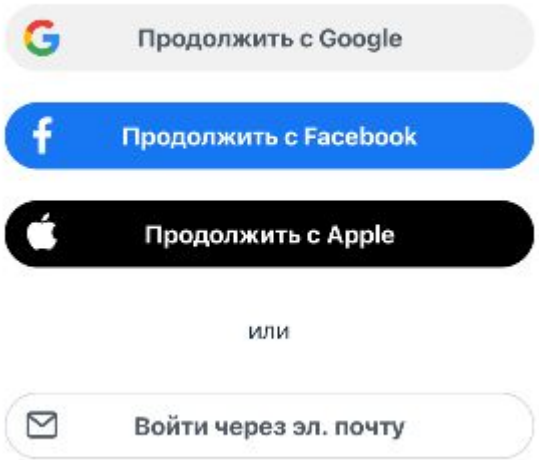
OAuth 2.0 и OpenID Connect

Как устроена кнопка «Войти через Google»

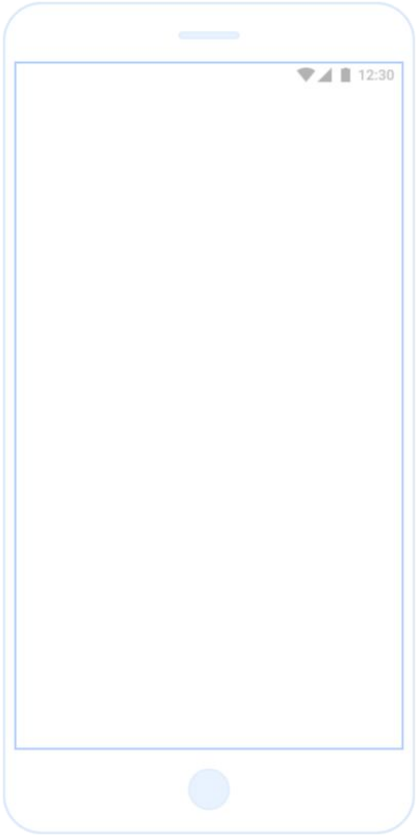


Выполнили: Сафиуллина Д.Р., Дорошенко А.В.

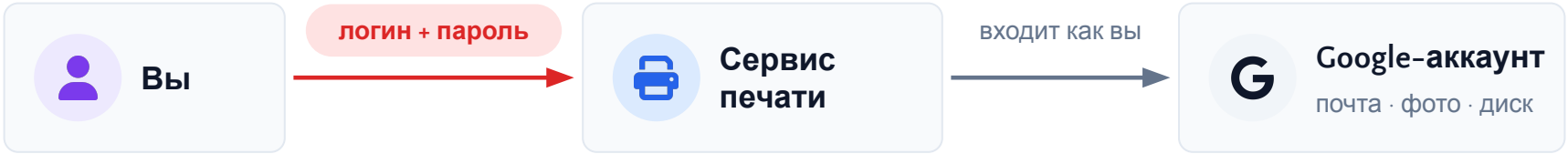
«Войти через Google»?



1. **Откуда сайт узнал моё имя и почту?**
2. **Получил ли он мой пароль от Google?**
3. **Как реализуется функция «разрешить доступ»?**



Доступ через пароль



Пароль у чужого сервиса

Взломают сервис — утечёт ваш главный аккаунт

Доступ ко всему

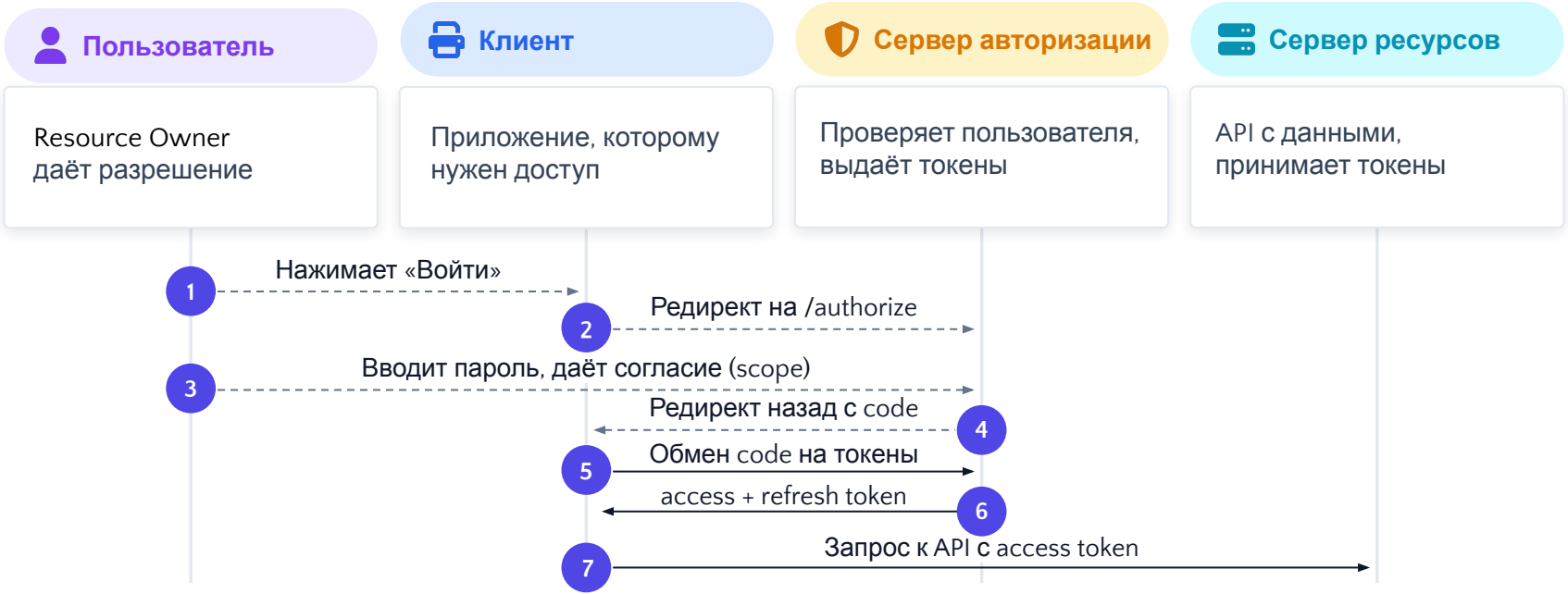
Нельзя разрешить «только фото»: видны и почта, и диск

Нельзя отозвать

Только смена пароля — и сломаются все остальные приложения

OAuth 2.0 (Authorization Code Flow)

Приложение получает не пароль, а **access token** — временный ключ с ограниченными правами (scope)



Области доступа, Access Token и Refresh Token

1 Области доступа (Scopes)

Определяют, какой доступ разрешён.

Запрошенные права

- Чтение календаря ☑
- Изменение календаря ☐
- Чтение контактов ☑

Принцип минимальных привилегий.
Запрашивайте только те права, которые действительно нужны приложению.

2 Access Token

Короткоживущий токен для вызова API.

Access Token
eyJhbGciOiJIUzI1NiIs...

Обычно живёт недолго

Используется для доступа к защищённым ресурсам.
Передаётся в запросах к API, например в заголовке Authorization.

3 Refresh Token

Долгоживущий токен для получения нового access token.

Refresh Token
def50200a4f2c1b7...

Обычно живёт дольше

Используется для получения нового access token.
Его нужно хранить безопасно.

Scopes ограничивают, что можно делать. | **Токены дают доступ к API.**

Типы клиентов и PKCE

Confidential

Есть бэкенд: client_secret хранится на сервере и никому не виден

Веб-приложение с сервером

Public

Код работает у пользователя — любой секрет из него можно извлечь

SPA, мобильное приложение, CLI

PKCE — Proof Key for Code Exchange

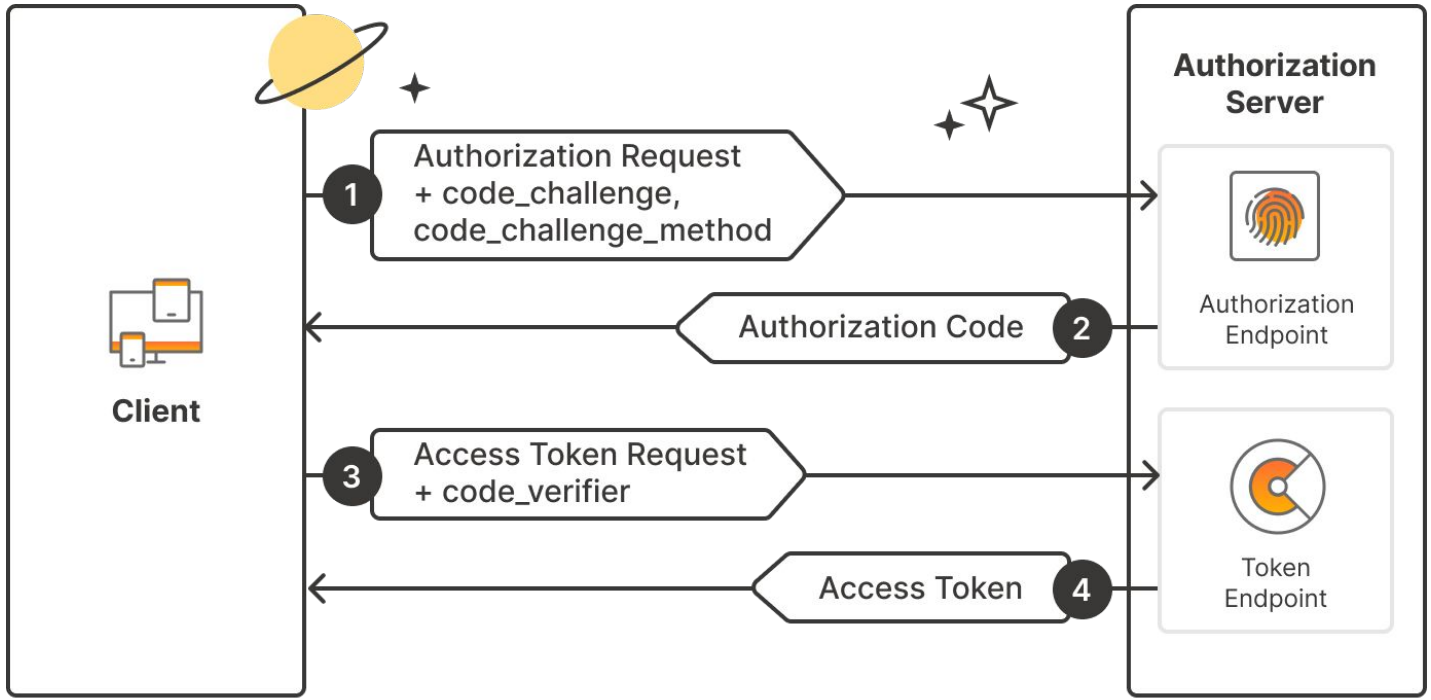
RFC 7636 · одноразовый секрет на каждый вход

`code_verifier` случайная строка → SHA-256 → `code_challenge` хеш — можно показывать

2 **Шаг 2:** при авторизации уходит только `code_challenge`

5 **Шаг 5:** при обмене кода на токен — сам `code_verifier`

✓ **Сервер:** `SHA-256(verifier) = challenge`


Client
Authorization Server
1
Authorization Request
+ code_challenge,
code_challenge_method
Authorization Endpoint
2
Authorization Code
3
Access Token Request
+ code_verifier
Token Endpoint
4
Access Token

Авторизация ≠ аутентификация

	OAuth 2.0 авторизация · «пропуск»	OpenID Connect аутентификация · «паспорт»
Отвечает на вопрос	«Что приложению разрешено?»	«Кто пользователь?»
Главный токен	Access token	ID Token (+ access token)
Кому адресован	Серверу ресурсов (API)	Клиенту (приложению)
Формат	Не определён в RFC 6749	Всегда JWT с подписью
Как включить	`scope=photos.read`	`scope=openid`

Безопасность: угроза → защита

Перехват authorization code

PKCE: без code_verifier код бесполезен

CSRF, подмена ответа

state в запросе; в OIDC — ещё и nonce

Подмена адреса возврата

точное совпадение redirect_uri

Утечка токена

HTTPS, короткий срок, ротация refresh

Поддельный ID Token

проверка подписи, iss, aud, exp

XSS крадёт токены в браузере

httpOnly-cookie или BFF, не localStorage

По RFC 9700 «Best Current Practice for OAuth 2.0 Security» (январь 2025)

Спасибо за внимание!